Master's Thesis — Preparatory Technical Report

Processing-in-Memory for Approximate Nearest Neighbour Search

A simulator-based study of why graph-based vector search resists near-memory acceleration

This document explains, from first principles, every concept behind the thesis: the memory-wall problem, Processing-in-Memory, vector search and its index structures, how DRAM timing works, how architecture simulators are used, and the experiment we ran on the Ramulator cycle-accurate simulator. It is written so that a technically literate reader with no prior exposure to any of these specific topics can follow the whole argument.

Artashes • Computer Science / Computer Architecture (HPC)

November 5, 2026

Contents

1 Executive summary **2**

2 The problem: the memory wall **2**
2.1 How a conventional computer is organised . . . 2
2.2 Why moving data is the expensive part . . . 2
2.3 Which workloads hit the wall . . . 3

3 Processing-in-Memory (PIM) **3**
3.1 The core idea . . . 3
3.2 Two families: PUM and PNM . . . 3
3.3 Real PIM hardware exists today . . . 3

4 Vector search and Approximate Nearest Neighbours **4**
4.1 What a vector is, and why we search over vectors . . . 4
4.2 Why "approximate" . . . 4
4.3 The three index structures: IVF, HNSW, PQ . . . 4

5 How DRAM timing really works **5**
5.1 Rows, columns, and the row buffer . . . 5
5.2 Why access pattern decides everything . . . 5

6 Simulators: how we study hardware we don't have **6**
6.1 Why simulate . . . 6
6.2 Ramulator . . . 6
6.3 The trace-driven workflow . . . 6

7 Methodology: the experiment we ran **6**
7.1 The workload and the two access patterns . . . 6
7.2 A teaching-size model first, then the real simulator . . . 7
7.3 The Ramulator configuration . . . 7

8 Results **7**
8.1 The numbers . . . 7
8.2 Reading the result . . . 8
8.3 Toy model versus real Ramulator . . . 8

9 Why this matters, and the research gap **8**
9.1 The literature splits exactly along this line . . . 8
9.2 What the result de-risks . . . 9

10 Which lever recovers the random case? Reordering vs. layout **9**
10.1 How we approached the problem: expectation vs. reality . . . 9
10.2 Negative result: access reordering does not help . . . 10
10.3 Positive result: data layout recovers locality completely . . . 10
10.4 How much layout is enough? . . . 10

11 Limitations (stated honestly) **10**

12 Conclusions and future work **11**
12.1 Conclusions . . . 11
12.2 Future work . . . 12

13 Reproducibility **12**

Glossary **12**

1 Executive summary

Modern computers spend a large and growing share of their time and energy *moving data* between memory and the processor rather than computing on it. For a whole class of important workloads — databases, machine learning, and *vector search* — this data movement, not the arithmetic, is the true bottleneck. **Processing-in-Memory (PIM)** is an architectural response: place computation *inside or beside* the memory, so the data barely has to move.

This thesis focuses on one high-demand, under-explored corner of that landscape: **Approximate Nearest Neighbour (ANN) search** — the engine behind recommendation systems, semantic search, and the retrieval step of modern AI (Retrieval-Augmented Generation). ANN is strongly memory-bound, which makes it a natural PIM target. However, the field has quickly converged on one *style* of ANN (cluster-based, called IVF/PQ), while a second, more powerful style (graph-based, called HNSW) remains almost untouched on PIM.

We argue, and then *measure*, *why*: graph-based search accesses memory in a random, pointer-chasing pattern that destroys the locality PIM depends on. Using a memory trace of a real ANN kernel, replayed through the peer-reviewed **Ramulator** cycle-accurate DRAM simulator with an HBM2 memory model, we show that merely changing the access pattern from sequential to random — with the algorithm and data held fixed — raises the cycle count by 41%, drops the DRAM row-buffer hit rate from 93.6% to 55.1%, and increases memory latency by 40%. This is the quantitative heart of the thesis direction, and it is fully reproducible from the accompanying software repository.

2 The problem: the memory wall

2.1 How a conventional computer is organised

A conventional computer separates **storage** from **computation**. The processor (CPU) contains the arithmetic units and a small amount of very fast memory (registers and caches). The bulk of the working data lives in **main memory**, built from **DRAM** (Dynamic Random-Access Memory) chips, which is much larger but much slower and physically separated from the CPU by a bus (a set of wires called the **memory channel**).

To perform any operation on a piece of data, the processor must first fetch it across that channel, place it in a register, compute, and often write the result back. This organisation — data is brought *to* the computation — is called **processor-centric** design, and it has been the dominant paradigm since the 1940s (it is sometimes called the *von Neumann* model).

2.2 Why moving data is the expensive part

The uncomfortable fact of modern hardware is that *moving* a byte costs far more than *computing* on it — in time, and especially in energy. A floating-point multiply might take a fraction of a nanosecond and a few picojoules; fetching the operands from off-chip DRAM can take tens of nanoseconds and hundreds of picojoules. As Mutlu et al. summarise in their widely cited survey *A Modern Primer on Processing in Memory*, measurements on real systems show that *62.7% of total system energy*, on average, is spent on data movement across the memory hierarchy; on mobile devices the figure is around 62%, and for large neural networks on edge accelerators more than 90% of energy goes to memory.

The memory wall. Processor speed has historically improved much faster than memory latency and bandwidth. The result is that, for data-intensive workloads, the processor spends most of its time *waiting* for data. Adding more cores or raising the clock frequency does not help if the bottleneck is feeding those cores. This growing gap is called the *memory wall*.

2.3 Which workloads hit the wall

A workload is **memory-bound** when its performance is limited by memory bandwidth or latency, and **compute-bound** when it is limited by arithmetic throughput. A useful shorthand is **arithmetic intensity** — the number of arithmetic operations performed per byte of data moved. Low arithmetic intensity means memory-bound.

The workloads that hit the wall hardest — graph analytics, databases, genome analysis, and much of machine learning — share this low arithmetic intensity: they stream through enormous amounts of data while doing very little computation per byte. Vector search, the subject of this thesis, is a textbook example.

3 Processing-in-Memory (PIM)

3.1 The core idea

If the problem is that data must travel to the computation, the obvious inversion is to move the computation to the data. **Processing-in-Memory** (PIM) places compute units *in or near* the memory — inside the DRAM chip, next to the memory banks, in the logic layer of 3D-stacked memory, or in the memory controller — so that computation happens where the data already lives and data movement is reduced or eliminated.

> **Analogy.** In the processor-centric model you drive to the library, check out every book you might need, drive home, read, and drive back. PIM puts a small desk *inside* the library, so you read where the books already are. Less driving means less time and energy wasted.

3.2 Two families: PUM and PNM

The literature classifies PIM designs by *how* and *where* they compute.

Processing-Using-Memory (PUM). Computation is performed using the memory array's own analog behaviour — the cells and sense amplifiers do the work. This is extremely area-efficient and keeps data perfectly in place, but it supports only a narrow set of specialised operations (e.g. bulk copy, bulk bitwise logic, bit-serial arithmetic). Representative mechanisms include RowClone, Ambit, and SIMDRAM.

Processing-Near-Memory (PNM). Conventional logic — even full small processors — is placed physically close to the memory. This is flexible (it can run general code) but data still moves from the array to that logic over internal buses. Representative real systems include UPMEM's general-purpose in-DRAM cores and Samsung's HBM-PIM SIMD units.

3.3 Real PIM hardware exists today

PIM is no longer only a research idea. **UPMEM** ships DRAM modules containing thousands of small general-purpose cores (DPUs, DRAM Processing Units). **Samsung** has demonstrated HBM-PIM ("Aquabolt-XL"), which embeds wide SIMD units inside a High-Bandwidth-Memory stack and reports, for example, an 8.9× speedup on matrix-vector multiplication. SK Hynix, Alibaba, and others have comparable efforts. These are PNM designs, and they are the concrete context for this thesis.

> **Our constraint.** We do *not* have access to PIM hardware. This is normal in academic work and is exactly what simulators are for (Section 6). It also shapes our choice of contribution: a *simulator-based* study that needs no special silicon.

4 Vector search and Approximate Nearest Neighbours

4.1 What a vector is, and why we search over vectors

In modern machine learning, a piece of content — a document, an image, a product, a user — is represented as an **embedding**: a list of numbers (a **vector**) of fixed length (the **dimension**, often 128 to 1536). The key property is that *semantic similarity becomes geometric closeness*: two documents about the same topic have vectors that are near each other in this high-dimensional space; unrelated documents are far apart.

Nearest Neighbour search is then the operation: given a query vector, find the stored vectors closest to it (by Euclidean or cosine distance). This is the retrieval engine behind recommendation ("items similar to this one"), semantic search, and **Retrieval-Augmented Generation (RAG)**, where a language model first retrieves relevant documents by vector similarity and then conditions its answer on them.

4.2 Why "approximate"

Computing the *exact* nearest neighbours means comparing the query against every stored vector. At the scale of modern systems — millions to billions of vectors — that is far too slow. So in practice we accept **Approximate Nearest Neighbour (ANN)** search: return vectors that are almost always the true nearest neighbours, in exchange for being orders of magnitude faster. The quality of an ANN method is measured by **recall** (what fraction of the true neighbours it returns) traded against **throughput** (queries per second).

A note on the acronym "ANN"

In this thesis, **ANN** always means *Approximate Nearest Neighbour* (search). The same three letters are sometimes used for *Artificial Neural Network*; that is a different concept and is never meant here.

4.3 The three index structures: IVF, HNSW, PQ

To avoid comparing against every vector, ANN systems build an **index** — a data structure that narrows the search. Three building blocks dominate, and the difference between them is central to this thesis.

IVF (Inverted File index). Cluster all vectors into, say, ten thousand groups using k-means. At query time, compute the distance to each cluster *centroid*, pick the few nearest clusters, and only scan the vectors inside those. This replaces a scan of everything with a scan of a few clusters. Its memory access is *sequential*: once a cluster is chosen, its vectors are read in order from a contiguous block.

PQ (Product Quantization). A *compression* technique usually combined with IVF. Each vector is split into sub-vectors, and each sub-vector is replaced by a one-byte code from a small learned codebook. A 512-dimension float vector (2 KB) shrinks to roughly 32–64 bytes, and distance computation becomes fast table lookups. The access pattern remains sequential.

HNSW (Hierarchical Navigable Small World). A *graph*-based index. Each vector is a node connected to its near neighbours, arranged in multiple layers. A query "walks" the graph, hopping greedily from node to node toward the query, coarse layer to fine. HNSW gives excellent recall and speed on a CPU — but its memory access is *random*: which node you visit next depends on distances you just computed, so you jump unpredictably around memory (*pointer-chasing*).

The crucial contrast. IVF/PQ reads memory *sequentially*; HNSW reads it *randomly*. Section 5 explains why this single difference decides whether PIM helps — and Section 8 measures it.

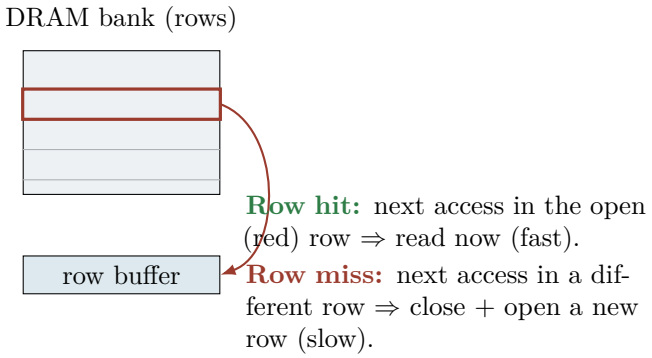
5 How DRAM timing really works

To understand the result, we need one piece of DRAM internals: the **row buffer**. This is the mechanism that turns "sequential vs. random access" into "fast vs. slow".

5.1 Rows, columns, and the row buffer

A DRAM chip stores bits in a two-dimensional grid of **rows** and **columns**, organised into independent **banks**. Crucially, you cannot read a single bit directly. To access any address, the memory controller must first **activate** its entire row, which copies that row into a fast holding area called the **row buffer**. Only then can columns be read from the buffer.

- If the *next* access is to the *same row* that is already in the buffer, it is a **row hit**: the data is read immediately. This is fast.
- If the next access is to a *different row*, it is a **row miss** (or *row conflict*): the controller must **precharge** (close) the current row and **activate** the new one before reading. This costs many extra cycles.



5.2 Why access pattern decides everything

Now the connection becomes mechanical:

- A **sequential scan** walks through addresses in order. Consecutive addresses almost always fall in the same DRAM row, so after the first activation you get a long run of *row hits*. The scan streams out of the open row at high speed.
- A **random traversal** jumps to unpredictable addresses. Each jump is likely to land in a different row, forcing a *row miss* — close, activate, read — again and again. Throughput collapses and latency rises.

This is why IVF/PQ (sequential) is friendly to high-bandwidth memory and PIM, while HNSW (random) is not. The *computation* is identical in both — the same distances over the same vectors — but the *order* of memory access determines the row-hit rate, and the row-hit rate determines performance. The thesis makes this precise and measures it.

6 Simulators: how we study hardware we don't have

6.1 Why simulate

Real PIM hardware is scarce, expensive, and — most importantly for research — *fixed*. We cannot ask physical silicon "what if the memory had twice the banks?" or "what if the access pattern changed?". A **computer-architecture simulator** is a software program that imitates hardware: you feed it a workload, and it reports how the hardware *would* behave — cycle counts, bandwidth, energy — on an ordinary computer. This lets us explore designs cheaply, and it is the standard methodology of the field.

Simulators differ in **fidelity**. A **functional** simulator computes the right *answer* but not realistic timing. A **cycle-accurate** (or cycle-level) simulator models behaviour clock-tick by clock-tick, including the DRAM timing constraints described above; it is detailed and trustworthy but slower to run. For a timing claim to be credible, a cycle-accurate model is required.

6.2 Ramulator

We use **Ramulator** (version 2.1), an open-source, cycle-accurate DRAM simulator written in C++20 and developed by the SAFARI research group at ETH Zürich (Onur Mutlu's group — the same authors as the PIM primer cited earlier). Ramulator models the memory controller and DRAM device for all current standards, including HBM1/2/3/4, the stacked memory where commercial PIM lives. It is **trace-driven**: you supply a list of memory accesses, and it replays them against its timing model, reporting cycles, row hits and misses, latency, and bandwidth. Because it is peer-reviewed and widely used, results obtained with it are credible to a committee or reviewer.

6.3 The trace-driven workflow

The methodology has three steps, and it cleanly separates the *workload* from the *hardware model*:

1. **Kernel.** Take (or write) the real computation — here, the inner loop of ANN distance search.
2. **Trace.** Record the sequence of memory addresses the kernel touches, as a list of `LD`/`ST` (load/store) records. This is the **memory trace**.
3. **Replay.** Feed the trace to Ramulator's cycle-accurate model and read out the timing statistics.

The advantage is modularity: we can swap the timing model (for example, from a simple teaching model to full Ramulator) *without touching the kernel or the trace*.

7 Methodology: the experiment we ran

7.1 The workload and the two access patterns

Our kernel is the inner loop of IVF/PQ-style ANN: for a batch of candidate vectors stored contiguously, compute the (squared Euclidean) distance from a query to each, and keep the k smallest. Concretely we use 4,096 candidate vectors of dimension 128, quantised to 8-bit integers (PQ style), which produces 8,192 memory reads in 64-byte bursts (one cache line each).

We generate *two* traces of this workload that differ only in the *order* of access:

Sequential — candidates read in storage order. This models an **IVF/PQ cluster scan**.

Random — candidates visited in a shuffled order. This models **graph/HNSW traversal**'s pointer-chasing.

Everything else — the vectors, their count, the bytes moved, the distance arithmetic — is held identical. Only the memory access order changes. This is a controlled experiment: it isolates *access pattern* as the single independent variable.

7.2 A teaching-size model first, then the real simulator

Before building a large simulator, we wrote a roughly 90-line C++ program — the **toy model** — that models only the one DRAM effect that dominates a scan: the row buffer. It reads a trace and charges a few cycles for a row hit and many more for a row miss, counting each. Its purpose is pedagogical and methodological: it lets us see the whole trace-driven flow in minutes and build intuition. Its absolute numbers are *illustrative ratios, not real nanoseconds*, because it ignores banks, refresh, and the scheduler. Once the method was clear, we handed the identical traces to real Ramulator for trustworthy numbers.

7.3 The Ramulator configuration

We configured Ramulator with its `LoadStoreTrace` front-end (which reads exactly our `LD`/`ST` trace format), an HBM2 device (8 Gb per channel, 2.0 Gbps), an `Open` row policy (which keeps a row open after access, rewarding sequential reuse and penalising random access — precisely the mechanism under study), and a first-ready, first-come-first-served (`FR-FCFS`) memory scheduler. The run is shown abbreviated below.

```
frontend = ramulator.frontend.LoadStoreTrace(clock_ratio=8, path=trace)
hbm2 = ramulator.dram.HBM2(org_preset="HBM2_8Gb",
                           timing_preset="HBM2_2000Mbps")
ctrl = ramulator.controller.GenericDDR(
    dram=hbm2, scheduler=ramulator.scheduler.FRFCFS(),
    row_policy=ramulator.row_policy.Open(), # the row-buffer effect
    addr_mapper=ramulator.addr_mapper.RoBaRaCoCh())
sim = ramulator.Simulation(frontend, mem); sim.run()
```

8 Results

8.1 The numbers

Table 1 reports the cycle-accurate HBM2 results for the two traces.

Metric	Sequential (IVF/PQ)	Random (graph/HNSW)	Change
Cycles	12,829	18,134	+41%
Row-hit rate	93.6%	55.1%	−38 pts
Avg. read latency	67.1 cyc	93.8 cyc	+40%
Read throughput	20,354 MB/s	14,394 MB/s	−29%
Row hit / miss / confl.	7639/225/296	4502/300/3365	—

Table 1: Cycle-accurate HBM2 results (Ramulator 2.1). Workload: 4,096 vectors, dimension 128, int8, 8,192 reads; HBM2_8Gb at 2.0 Gbps, Open row policy, FR-FCFS scheduler.

8.2 Reading the result

The computation was identical in both runs — the same vectors, the same number of reads, the same arithmetic. The *only* change was the order of memory access. Yet switching from sequential to random:

- dropped the DRAM row-hit rate from **93.6%** to **55.1%** (a 38-point fall);
- raised the cycle count by **41%**;
- raised average read latency by **40%** and cut sustained throughput by **29%**.

The row-miss-plus-conflict count tells the mechanical story directly: the sequential run suffered only $225 + 296$ row conflicts out of 8,192 reads, while the random run suffered $300 + 3365 \approx 3665$ — more than a tenfold increase in the expensive case. This is exactly the row-buffer behaviour predicted in Section 5, now confirmed on a peer-reviewed, cycle-accurate simulator.

8.3 Toy model versus real Ramulator

The teaching model and the real simulator agree on the *shape* of the result (sequential far more row-hit-friendly than random) but differ in the exact numbers, as expected (Table 2). Real HBM2 has many banks and an intelligent FR-FCFS scheduler, which recover some row hits that the toy's single-row model misses. This is precisely why one reports the simulator numbers, not the toy's: the toy builds intuition; Ramulator provides evidence.

	Toy model	**Real Ramulator (HBM2)**
Sequential row-hit	99.2%	93.6%
Random row-hit	50.7%	55.1%
Fidelity	illustrative ratios	cycle-accurate, validated

Table 2: The toy model agrees with Ramulator on the qualitative shape; Ramulator provides the trustworthy quantitative numbers.

9 Why this matters, and the research gap

9.1 The literature splits exactly along this line

A survey of recent work (2024–2025) shows the field has produced several ANN-on-PIM systems — DRIM-ANN, UpANNS, and PIMANN among them — all of which accelerate the *cluster-based* IVF/PQ style on real UPMEM hardware, reporting speedups in the range of roughly 2.5–4.3× over a CPU. Every one of them keeps memory access sequential, which (as we have just measured) is what makes PIM effective. By contrast, *graph-based* ANN (HNSW, DiskANN) on PIM is almost absent from the literature — precisely because its random, data-dependent traversal is hostile to the row-buffer locality that PIM relies on.

The gap, stated precisely. Cluster-based ANN on PIM is becoming crowded and works because it is sequential. Graph-based ANN on PIM is open because it is random. Our experiment turns that observation from an assertion into a measured fact, and provides a concrete baseline against which any locality-improving technique can be evaluated.

9.2 What the result de-risks

Beyond the scientific point, the experiment establishes that the entire toolchain works end to end: a real cycle-accurate simulator builds from source, our ANN traces run through it, and sensible numbers come out. The research method is proven *before* the hard, open research begins. For a thesis, that is a significant de-risking.

10 Which lever recovers the random case? Reordering vs. layout

Characterising the problem is only half the work; the thesis contribution is to *recover* the lost performance. The obvious idea is to *reorder* accesses so that same-row requests are serviced together. We tested it on the real simulator — and found that it does *not* work. The intervention that does work is changing the data *layout*. This section reports both, because the negative result is as instructive as the positive one.

10.1 How we approached the problem: expectation vs. reality

It is worth recording the actual path of this investigation, because the route to the answer is itself part of the contribution — and because it is a small case study in letting measurement overrule intuition.

Our initial hypothesis. Having established that the random access pattern destroys row-buffer locality (Chapter 8), the natural first instinct was: *if the problem is that accesses arrive in a locality-hostile order, then re-ordering them should fix it.* Every textbook lever for improving DRAM locality — query batching, a deeper memory-controller reorder window, software access scheduling — is ultimately a way of reordering pending requests so that same-row accesses are serviced together. We expected that sorting the random access stream within a window would recover a large fraction of the lost locality, smoothly approaching the sequential ideal as the window grew.

What the teaching model predicted. Our first, teaching-size timing model (which tracks only a single open DRAM row) agreed enthusiastically. It predicted that windowed reordering would lift the row-hit rate from 50.7% toward 78.0% as the window reached a few hundred accesses — a clean, monotone recovery curve. On the strength of that, we tentatively wrote up "access reordering partly recovers the gap" as the proposed contribution.

What actually happened. When we replayed the *same* reordered traces through the cycle-accurate Ramulator simulator with a real HBM2 model, the recovery *vanished*. The row-hit rate barely moved — from 55.1% to 56.2% across the entire window sweep. The teaching model had been wrong, and the real simulator corrected us. This was initially surprising; on reflection it is obvious. A real memory controller already uses a first-ready (FR-FCFS) scheduler that reorders its own request queue to prioritise row hits. Our software pre-sorting was simply *doing again, less well, what the hardware already does.* Reordering is redundant, and therefore useless, against a modern controller.

The pivot that followed. The failure was informative: it told us the lever we needed must be something the controller *cannot* do for us. A controller can reorder *when* it reads addresses, but it cannot change *where* the data physically lives. That reframing pointed directly at *data layout* — placing co-visited vectors in the same DRAM rows — which we then tested and found to recover the locality completely. The remainder of this chapter reports both results in order: first the reordering null result, then the layout result that it led us to.

10.2 Negative result: access reordering does not help

We took the random (graph/HNSW) trace and sorted its accesses within sliding windows of size W before replaying them through cycle-accurate HBM2 (W=1 is untouched random; larger W approaches a sorted, sequential order). The row-hit rate barely moves: from 55.1% at W=1 to only 56.2% at W=256 (Table 3). The reason is architectural. The HBM2 memory controller already uses a *first-ready, first-come-first-served* (FR-FCFS) scheduler, which reorders its own request queue to prioritise row hits. Our software pre-sorting is therefore *redundant with work the controller already performs*, and cannot beat it. **Access reordering is the wrong lever.**

Reorder window W	Row-hit rate (real HBM2)
1 (pure random)	55.1%
32	55.1%
64	55.1%
128	55.3%
256	56.2%

Table 3: Reordering the random trace within a window W recovers almost nothing on real Ramulator HBM2 — the FR-FCFS controller already reorders for row hits, so software pre-sorting is redundant.

10.3 Positive result: data layout recovers locality completely

The one thing the controller *cannot* do is change *where* data lives. We store the candidate vectors so that those visited together (members of the same coarse cluster, or graph neighbours) occupy contiguous addresses — hence the same DRAM rows — while still visiting clusters in random order, exactly as graph traversal does. This *cluster-ordered layout* recovers the locality in full: on real HBM2 it reaches **93.6% row-hit and 13,097 cycles, statistically identical to the sequential baseline** (93.6%, 12,829 cycles). The access pattern is still random at the graph level; only the placement changed.

10.4 How much layout is enough?

Sweeping the cluster size C (Table 4) shows the recovery is steep and saturates early. Clusters of just C=4 vectors already reach 87.6% row-hit; by C=16 the rate hits the sequential ceiling of 93.6% and stays there. The practical message is strong and cheap: **co-locating only about sixteen co-visited vectors is enough to make graph-ANN read memory exactly as well as a sequential scan** — a one-time, offline index-build transformation.

> This is the heart of the contribution. The naive fix (reordering) fails because the memory controller already does it; the real fix is *data layout*, which gives the controller row-hits it could not otherwise obtain. The thesis proper would implement cluster-ordered placement inside a real graph index (e.g. HNSW over SIFT1M) and measure end-to-end recall and throughput.

11 Limitations (stated honestly)

A result is only as useful as its caveats are clear.

Cluster size C	Row-hit rate (real HBM2)
1 (pure random)	55.4%
4	87.6%
16	93.6%
32	93.6%
64	93.6%
sequential ceiling	*93.6%*

Table 4: Cluster-ordered layout recovers locality on real HBM2, saturating at the sequential ceiling by a cluster size of only $C \approx 16$ vectors.

DRAM path only. Ramulator models the memory timing, not a near-memory compute unit. We therefore report the *locality degradation* (row-hit rate, latency, cycles), not a direct host-versus-PIM speedup. Producing a true PIM speedup number requires additionally modelling a near-bank compute datapath.

Static trace. Real graph traversal is *data-dependent*: the next node visited depends on distances computed at run time. Our random trace *approximates* that irregularity; a fully faithful study would generate per-query traces from an executing HNSW index.

Single configuration. We report one dataset size and one HBM2 preset. We separately showed, on the toy model, that the effect is independent of dataset size; re-running that sweep on Ramulator is a near-term next step.

Not yet hardware-validated. Ramulator itself is validated against real DRAM, but our specific ANN-to-memory mapping should, where possible, be cross-checked against a measurement on a real UPMEM or HBM-PIM system.

12 Conclusions and future work

12.1 Conclusions

1. ANN distance search is memory-bound: the arithmetic is trivial, so performance is governed by how data moves. This makes it a natural PIM target.

2. The DRAM *row buffer* is the deciding mechanism. A high row-hit rate (sequential access) yields fast memory; a low row-hit rate (random access) yields slow memory.

3. *Access pattern*, not the computation and not the dataset size, determines whether PIM helps. We demonstrated this with a controlled experiment in which only the access order changed, and performance degraded by 41% in cycles and 38 points in row-hit rate.

4. The penalty is recoverable — but by *layout*, not reordering. Software access reordering barely helps (55.1%→56.2%) because the FR-FCFS controller already reorders; cluster-ordered *data layout* fully recovers locality (to 93.6%, the sequential ceiling) with clusters of only ∼16 vectors (Section 10).

5. This explains the shape of the literature: cluster-based (sequential) ANN on PIM succeeds and is crowding; graph-based (random) ANN on PIM is the genuinely open problem.

12.2 Future work

The natural research programme follows directly:

- **Sweep on Ramulator.** Reproduce the size- and dimension-independence of the effect with cycle-accurate numbers (the first quantitative figure of the thesis proper).
- **Model near-memory compute.** Add a near-bank datapath to obtain a true host-vs-PIM speedup, not just a locality metric.
- **Faithful graph traces.** Instrument a real HNSW search to emit per-query memory traces, replacing the random approximation.
- **Attack the gap.** Design and evaluate locality-improving techniques for graph-based ANN — data layout, node clustering, traversal scheduling — and measure whether they recover the row-hit rate (and hence the PIM benefit). Section 10 already identifies the correct lever — cluster-ordered data layout, which fully recovers locality on real HBM2 — and rules out access reordering; the thesis proper would implement that layout inside a real graph index (HNSW over SIFT1M) and measure end-to-end recall and throughput. This is the core contribution the thesis would make.

13 Reproducibility

Every number in this document regenerates from the accompanying software repository (`Thesis-PIM-ANN`). The repository contains the ANN kernel, the trace generator (which emits Ramulator's `LD`/`ST` format), the teaching toy model, the reordering tool, and a verified build-and-run script. On a Linux machine with a C++20 compiler and Python 3.10:

```
git clone Thesis-PIM-ANN.bundle Thesis-PIM-ANN
cd Thesis-PIM-ANN
make demo # Experiments 1-2 (sequential vs random)
make optsweep # Section 11 (windowed-reorder recovery)
./scripts/setup_ramulator.sh # builds Ramulator 2.1 and runs on real HBM2
```

The script downloads a prebuilt CMake if needed, clones and builds Ramulator (pre-fetching its dependencies for a robust offline build), generates the ANN traces, and replays them through the cycle-accurate HBM2 model — reproducing Table 1.

Glossary

ANN Approximate Nearest Neighbour search — find vectors close to a query, approximately.

Arithmetic intensity Operations performed per byte of data moved; low ⇒ memory-bound.

Bank An independently accessible sub-array within a DRAM chip.

Cycle-accurate A simulator that models behaviour clock-tick by clock-tick, including real timing.

DPU DRAM Processing Unit — UPMEM's small general-purpose in-memory core.

DRAM Dynamic Random-Access Memory — the technology of main memory.

Embedding A fixed-length vector representing content, where closeness means similarity.

HBM High-Bandwidth Memory — fast 3D-stacked DRAM used in GPUs/AI accelerators and PIM.

HNSW Hierarchical Navigable Small World — a graph-based ANN index (random access).

IVF Inverted File index — a cluster-based ANN index (sequential access).

Memory-bound Limited by memory bandwidth/latency rather than by arithmetic.

PIM Processing-in-Memory — computing in or near memory to cut data movement.

PNM / PUM Processing-Near-Memory (logic beside memory) / Processing-Using-Memory (compute in the array).

PQ Product Quantization — compresses vectors into small codes for fast ANN.

RAG Retrieval-Augmented Generation — an LLM that retrieves documents (via ANN) before answering.

Row buffer The fast holding area for an activated DRAM row; hits are fast, misses slow.

Row hit / miss Next access in the open row (fast) / a different row (slow).

Trace-driven A simulation method that replays a recorded list of memory accesses.